```
!pip install -q langchain langgraph langchain-community langchain-core
!pip install -q pypdf python-dotenv
!pip install -q groq langchain_openai
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.5/2.5 MB 29.2 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.0/1.0 MB 47.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 65.0/65.0 kB 4.8 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 51.0/51.0 kB 3.6 MB/s eta 0:00:00
ERROR: pip's dependency resolver does not currently take into account all the packages that are insta
google-colab 1.0.0 requires requests==2.32.4, but you have requests 2.33.0 which is incompatible.
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 333.7/333.7 kB 6.4 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 139.7/139.7 kB 3.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 88.5/88.5 kB 5.7 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 506.8/506.8 kB 16.6 MB/s eta 0:00:00
```

```
import os
os.environ["GROQ_API_KEY"] = "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"
```

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(
    model="llama-3.3-70b-versatile",
    openai_api_key=os.environ["GROQ_API_KEY"],
    base_url="https://api.groq.com/openai/v1",
    temperature=0
)

print("LLM Ready")
```

```
LLM Ready
```

```
SYSTEM_PROMPT = """
You are a document processing specialist.

Tasks:
1. Classify document:
   - Cease
   - Irrelevant
   - Uncertain

2. Extract:
   - Name
   - Date
   - Request reason

3. Provide confidence score.

Rules:
- Be precise
- If unclear → Uncertain
- Maintain structured output
"""
```

```
from google.colab import files

uploaded = files.upload()
```

```
Choose files 19 files
notice_5.pdf(application/pdf) - 530647 bytes, last modified: 23/01/2026 - 100% done
LoA1.pdf(application/pdf) - 311776 bytes, last modified: 23/01/2026 - 100% done
notice_4.pdf(application/pdf) - 534156 bytes, last modified: 23/01/2026 - 100% done
notice_2.pdf(application/pdf) - 543574 bytes, last modified: 23/01/2026 - 100% done
notice_3.pdf(application/pdf) - 549749 bytes, last modified: 23/01/2026 - 100% done
notice_1.pdf(application/pdf) - 562221 bytes, last modified: 23/01/2026 - 100% done
bw_doc_5.pdf(application/pdf) - 288554 bytes, last modified: 23/01/2026 - 100% done
bw_doc_4.pdf(application/pdf) - 284236 bytes, last modified: 23/01/2026 - 100% done
bw_doc_2.pdf(application/pdf) - 274613 bytes, last modified: 23/01/2026 - 100% done
bw_doc_3.pdf(application/pdf) - 285612 bytes, last modified: 23/01/2026 - 100% done
bw_doc_1.pdf(application/pdf) - 312773 bytes, last modified: 23/01/2026 - 100% done
LOA9.pdf(application/pdf) - 1364891 bytes, last modified: 23/01/2026 - 100% done
LOA8.pdf(application/pdf) - 1139882 bytes, last modified: 23/01/2026 - 100% done
LOA6.pdf(application/pdf) - 791380 bytes, last modified: 23/01/2026 - 100% done
LOA7.pdf(application/pdf) - 813193 bytes, last modified: 23/01/2026 - 100% done
LOA4.pdf(application/pdf) - 319210 bytes, last modified: 23/01/2026 - 100% done
LOA5.pdf(application/pdf) - 269051 bytes, last modified: 23/01/2026 - 100% done
LOA3.pdf(application/pdf) - 367412 bytes, last modified: 23/01/2026 - 100% done
LOA2.pdf(application/pdf) - 311914 bytes, last modified: 23/01/2026 - 100% done
Saving notice_5.pdf to notice_5.pdf
Saving LoA1.pdf to LoA1.pdf
Saving notice_4.pdf to notice_4.pdf
Saving notice_2.pdf to notice_2.pdf
Saving notice_3.pdf to notice_3.pdf
Saving notice_1.pdf to notice_1.pdf
Saving bw_doc_5.pdf to bw_doc_5.pdf
Saving bw_doc_4.pdf to bw_doc_4.pdf
Saving bw_doc_2.pdf to bw_doc_2.pdf
Saving bw_doc_3.pdf to bw_doc_3.pdf
Saving bw_doc_1.pdf to bw_doc_1.pdf
Saving LOA9.pdf to LOA9.pdf
Saving LOA8.pdf to LOA8.pdf
Saving LOA6.pdf to LOA6.pdf
Saving LOA7.pdf to LOA7.pdf
Saving LOA4.pdf to LOA4.pdf
Saving LOA5.pdf to LOA5.pdf
Saving LOA3.pdf to LOA3.pdf
Saving LOA2.pdf to LOA2.pdf
```

```
import os

os.listdir("/content")
```

```
['.config',
 'LOA7.pdf',
 'bw_doc_4.pdf',
 'bw_doc_2.pdf',
 'LOA3.pdf',
 'bw_doc_1.pdf',
 'LOA9.pdf',
 'LOA6.pdf',
 'LOA2.pdf',
 'bw_doc_5.pdf',
 'LoA1.pdf',
 'notice_2.pdf',
 'LOA4.pdf',
 'notice_1.pdf',
 'LOA8.pdf',
 'bw_doc_3.pdf',
 'notice_5.pdf',
 'LOA5.pdf',
 'notice_4.pdf',
 'notice_3.pdf',
 'sample_data']
```

```
from pypdf import PdfReader

def load_document(file_path):
    reader = PdfReader(file_path)
    text = ""

    for page in reader.pages:
        text += page.extract_text() or ""

    return text
```

```
def process_all_documents():

    import os

    pdf_files = [f for f in os.listdir("/content") if f.endswith(".pdf")]

    print("Files detected:", pdf_files)

    for file in pdf_files:

        file_path = f"/content/{file}"

        process_all_document(file_path)
```

```
import json
import re

def extract_json_safe(text):

    try:
        return json.loads(text)

    except:
        match = re.search(r"\{.*\}", text, re.DOTALL)

        if match:
            try:
                return json.loads(match.group())
            except:
                return None

    return None
```

```
def classify_document(text):

    prompt = f"""
    You are a document classification expert.

    Classify this document into one of:
    - Cease
    - Irrelevant
    - Uncertain

    Return JSON only:

    {{
        "label": "",
        "confidence": 0,
        "reason": ""
    }}

    Document:
    {text}
    """

    response = llm.invoke(prompt).content

    return extract_json_safe(response)
```

```
def extract_information(text):

    prompt = f"""
    You are an expert document extraction system.

    Extract the following details.

    Return STRICT JSON ONLY:

    {{
        "name": null or string,
        "date": null or string,
        "reason": null or string,
        "contact": null or string
```

```
    }}

    Document:
    {text}
    """

    response = llm.invoke(prompt).content

    return extract_json_safe(response)
```

```
import sqlite3

conn = sqlite3.connect("cease_requests.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS cease_requests (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    document_name TEXT,
    processed_date TEXT,
    extracted_data TEXT
)
""")

conn.commit()

print("Database created successfully")
```

```
Database created successfully
```

```
def store_to_database(document_name, extracted_data):

    cursor.execute(
        "INSERT INTO cease_requests (document_name, processed_date, extracted_data) VALUES (?, datetim
        (document_name, json.dumps(extracted_data))
    )

    conn.commit()
```

```
def audit_log(action, document_name):

    with open("audit_log.txt", "a") as f:
        f.write(f"{action} : {document_name}\n")
```

```
def human_review(text):

    print("\n⚠️ HUMAN REVIEW REQUIRED")
    print(text[:500])

    decision = input("Enter decision (Cease / Irrelevant): ")

    return decision
```

```
from langgraph.graph import StateGraph

from typing import TypedDict

class State(TypedDict):
    text: str
    document_name: str
    classification: dict

def classify_node(state):

    # Safety check (so KeyError will never happen again)
    if "text" not in state:
        print("ERROR: text not found in state")
        print("State received:", state)
        return state

    result = classify_document(state["text"])
```

```
    if result is None:
        result = {
            "label": "Uncertain",
            "confidence": 0,
            "reason": "Parsing failed"
        }

    state["classification"] = result

    return state


def route_node(state):

    label = state["classification"]["label"]

    if label == "Cease":
        return "cease"

    elif label == "Irrelevant":
        return "irrelevant"

    else:
        return "uncertain"


def cease_node(state):

    extracted_data = extract_information(state["text"])

    store_to_database(state["document_name"], extracted_data)

    audit_log("CEASE_DOCUMENT", state["document_name"])

    print("Stored in DB:", state["document_name"])

    return state


def irrelevant_node(state):

    audit_log("IRRELEVANT_DOCUMENT", state["document_name"])

    print("Irrelevant:", state["document_name"])

    return state


def uncertain_node(state):

    decision = human_review(state["text"])

    if decision == "Cease":
        return cease_node(state)
    else:
        return irrelevant_node(state)
```

```
graph = StateGraph(State)

graph.add_node("classify", classify_node)
graph.add_node("cease", cease_node)
graph.add_node("irrelevant", irrelevant_node)
graph.add_node("uncertain", uncertain_node)

graph.set_entry_point("classify")

graph.add_conditional_edges(
    "classify",
    route_node,
    {
        "cease": "cease",
        "irrelevant": "irrelevant",
        "uncertain": "uncertain"
    }
```

```
)

app = graph.compile()

print("LangGraph Ready")
```

```
LangGraph Ready
```

```
import os

def process_single_document(file_path):

    # Step 1: Load text from PDF
    text = load_document(file_path)

    # Step 2: Extract file name
    doc_name = os.path.basename(file_path)

    # Step 3: Create correct state (VERY IMPORTANT)
    state = {
        "text": text,
        "document_name": doc_name
    }

    # Step 4: Run LangGraph
    app.invoke(state)

    print("Completed:", doc_name)
```

```
import os

def process_all_documents():

    pdf_files = [f for f in os.listdir("/content") if f.endswith(".pdf")]

    print("Files detected:", pdf_files)

    for file in pdf_files:

        file_path = f"/content/{file}"

        print("\nProcessing:", file)

        # Step 1: Read PDF
        text = load_document(file_path)

        # Step 2: Create proper state
        state = {
            "text": text,
            "document_name": file
        }

        print("State created successfully")

        # Step 3: Run graph correctly
        app.invoke(state)

        print("Completed:", file)
```

```
app = graph.compile()
```

```
process_all_documents()
```

```
Files detected: ['LOA7.pdf', 'bw_doc_4.pdf', 'bw_doc_2.pdf', 'LOA3.pdf', 'bw_doc_1.pdf', 'LOA9.pdf',

Processing: LOA7.pdf
State created successfully
Stored in DB: LOA7.pdf
Completed: LOA7.pdf

Processing: bw_doc_4.pdf
State created successfully
Irrelevant: bw_doc_4.pdf
```

```
Completed: bw_doc_4.pdf

Processing: bw_doc_2.pdf
State created successfully
Irrelevant: bw_doc_2.pdf
Completed: bw_doc_2.pdf

Processing: LOA3.pdf
State created successfully
Stored in DB: LOA3.pdf
Completed: LOA3.pdf

Processing: bw_doc_1.pdf
State created successfully
Irrelevant: bw_doc_1.pdf
Completed: bw_doc_1.pdf

Processing: LOA9.pdf
State created successfully
Stored in DB: LOA9.pdf
Completed: LOA9.pdf

Processing: LOA6.pdf
State created successfully
Stored in DB: LOA6.pdf
Completed: LOA6.pdf

Processing: LOA2.pdf
State created successfully
Stored in DB: LOA2.pdf
Completed: LOA2.pdf

Processing: bw_doc_5.pdf
State created successfully
Irrelevant: bw_doc_5.pdf
Completed: bw_doc_5.pdf

Processing: LoA1.pdf
State created successfully
Stored in DB: LoA1.pdf
Completed: LoA1.pdf

Processing: notice_2.pdf
State created successfully
Irrelevant: notice_2.pdf
Completed: notice_2.pdf
```

```
cursor.execute("SELECT * FROM cease_requests")

rows = cursor.fetchall()

for row in rows:
    print(row)
```

```
(1, 'LOA7.pdf', '2026-03-25 17:18:59', '{"name": null, "date": null, "reason": null, "contact": "LAW (
(2, 'LOA3.pdf', '2026-03-25 17:19:00', '{"name": "BELLINA DAVANZO AMOEDO, RADCLIFF", "date": "6/29/202
(3, 'LOA9.pdf', '2026-03-25 17:19:02', '{"name": "RANDLE L WIDHALM", "date": "6/30/2025", "reason": nu
(4, 'LOA6.pdf', '2026-03-25 17:19:10', '{"name": "Amedo, Radcliff, Bellina Davanzo", "date": null, "re
(5, 'LOA2.pdf', '2026-03-25 17:19:15', '{"name": "LOVETTA CANNUNZIATA", "date": "7/2/2025", "reason":
(6, 'LoA1.pdf', '2026-03-25 17:19:21', '{"name": "DAVANZO.BELLINA AMOEDO,RADCLIFF", "date": "6/29/2025
(7, 'LOA4.pdf', '2026-03-25 17:19:28', '{"name": "DAVANZO, BELLINA AMOEDO, RADCLIFF", "date": "6/29/20
(8, 'LOA8.pdf', '2026-03-25 17:19:41', '{"name": "GARDER, BOBBI R", "date": "7/1/12025", "reason": nu
(9, 'LOA5.pdf', '2026-03-25 17:19:49', '{"name": "RANDLE L WIDHALM", "date": "6/30/2025", "reason": "
```